CASO

BELMONTE

RESUMEN DEL CASO

Desarrollar una plataforma móvil para gestionar solicitudes de traslados corporativos, permitiendo registrar viajes, asignarlos a conductores, consultar los viajes asignados, actualizar sus estados y abrir la ruta correspondiente en un mapa.

PROBLEMA ACTUAL

Actualmente, la operación se gestiona manualmente mediante:

LLAMADAS + CORREOS + WHATSAPP

Por estos canales circulan datos importantes como pasajeros, direcciones, vuelos, fechas y horarios. Esto provoca información dispersa, lentitud y riesgo de errores operacionales.

MEJORAS PROPUESTAS

El proyecto busca desarrollar un MVP que centralice digitalmente el ciclo de los traslados, desde la solicitud hasta la ejecución por parte del conductor.

PERFILES

01

Solicitante / Pasajero

Solicita un traslado indicando:

- fecha;
- hora;
- origen;
- destino;
- número de vuelo, cuando corresponda.

02

Administrador / Recepcionista

El anexo lo identifica como responsable de **tomar y distribuir las solicitudes**. Por lo tanto, para el proyecto semestral tiene sentido que pueda gestionar y asignar los viajes.

03

Conductor

Debe visualizar sus viajes asignados, consultar sus datos, cambiar el estado del servicio y acceder directamente a una aplicación de navegación.

Este último perfil es especialmente relevante porque la aplicación está concebida para uso intensivo en terreno.

FLUJO PRINCIPAL

SOLICITANTE

→ Crea solicitud de traslado → **Estado PENDIENTE**

ADMINISTRADOR

→ Consulta solicitudes pendientes → Selecciona conductor → Asigna traslado → **Estado ASIGNADO**

CONDUCTOR

→ Inicia sesión → Consulta sus viajes → Selecciona viaje → Consulta origen/destino → Inicia viaje → **Estado EN_CURSO** → Abre aplicación de mapas → Realiza traslado → Finaliza viaje → **Estado FINALIZADO**

FUNCIONALIDADES PRINCIPALES DEL SISTEMA

Autenticación y usuarios

Como requisito académico adicional, incorporaría:

- registro de usuario;
- login;
- Spring Security;
- JWT;
- almacenamiento seguro de contraseña con BCrypt;
- autorización según rol;
- endpoints protegidos.

Gestión de solicitudes de traslado

El solicitante podrá registrar:

- pasajero;
- fecha;
- hora;
- origen;
- destino;
- número de vuelo opcional;
- coordenadas de origen;
- coordenadas de destino.

La estructura base exigida por el documento incluye ID, pasajero, fecha/hora, origen y destino georreferenciados y estado.

Gestión de viajes

El administrador debería poder:

- consultar solicitudes;
- consultar detalle;
- asignar conductor;
- modificar asignación;
- visualizar estado;
- consultar viajes por fecha.

Funciones del conductor

El conductor debería poder:

- consultar viajes asignados;
- consultar viajes del día;
- ver detalle;
- identificar origen y destino;
- iniciar traslado;
- finalizar traslado;
- abrir navegación.

INTEGRACIÓN CON GOOGLE MAPS / WAZE

Es una de las características diferenciadoras de este proyecto. El conductor debería poder presionar, por ejemplo:

NAVEGAR AL ORIGEN *o* NAVEGAR AL DESTINO

y Android utilizaría un **Intent** para abrir Google Maps o Waze con las coordenadas correspondientes.

El requerimiento establece explícitamente esta integración con aplicaciones externas de navegación.

5. ARQUITECTURA TECNOLÓGICA EXIGIDA

CAPA	TECNOLOGÍA
Plataforma	**Android**
Lenguaje	**Kotlin**
IDE	**Android Studio**
UI	**Jetpack Compose**
Diseño	**Material Design 3**
Arquitectura app	**MVVM**
Persistencia local	**Room / SQLite**
Backend	**Spring Boot (puede ser otra tecnología)**
Comunicación	**API REST**
Cliente HTTP	**Retrofit**
Pruebas	**Pruebas unitarias**
Repositorio	**Git / GitHub**
Gestión	**Trello / Jira (puede ser otra)**
Entregable	**APK firmado**

6. FLUJO FUNCIONAL QUE PROPONDRÍA PARA EL MVP

Inicio de sesión (JWT)

↓

Menú según rol

↓

Crear solicitud de traslado

↓

POST API REST → Estado PENDIENTE

↓

Administrador: listado de solicitudes pendientes

↓

Asignar conductor → Estado ASIGNADO

↓

Conductor: viajes asignados / del día

↓

Detalle del viaje (origen, destino, vuelo)

↓

Iniciar viaje → Estado EN_CURSO

↓

Navegar al origen / destino (Intent → Google Maps / Waze)

↓

Finalizar viaje → Estado FINALIZADO

Paralelamente:

Viajes Asignados → Api Rest y:

Sesión / token → Room

CONSIDERACIONES IMPORTANTES

Autenticación

El documento no menciona creación de cuenta ni autenticación, pero si deberían estar consideradas.

"Tiempo real"

El documento utiliza expresiones como **actualizaciones en tiempo real**, pero no especifica WebSockets, Firebase, SSE u otra tecnología.

Para este proyecto no obligaría a implementar WebSockets. Una actualización mediante llamadas REST al ingresar/refrescar la pantalla sería suficiente, salvo que quieran incorporar tiempo real como desafío adicional.

Google Maps/Waze

No se obligará integrar un SDK completo de mapas.

El caso específicamente permite trabajar muy bien:

`Android Intent` → `aplicación externa` → `coordenadas origen/destino`

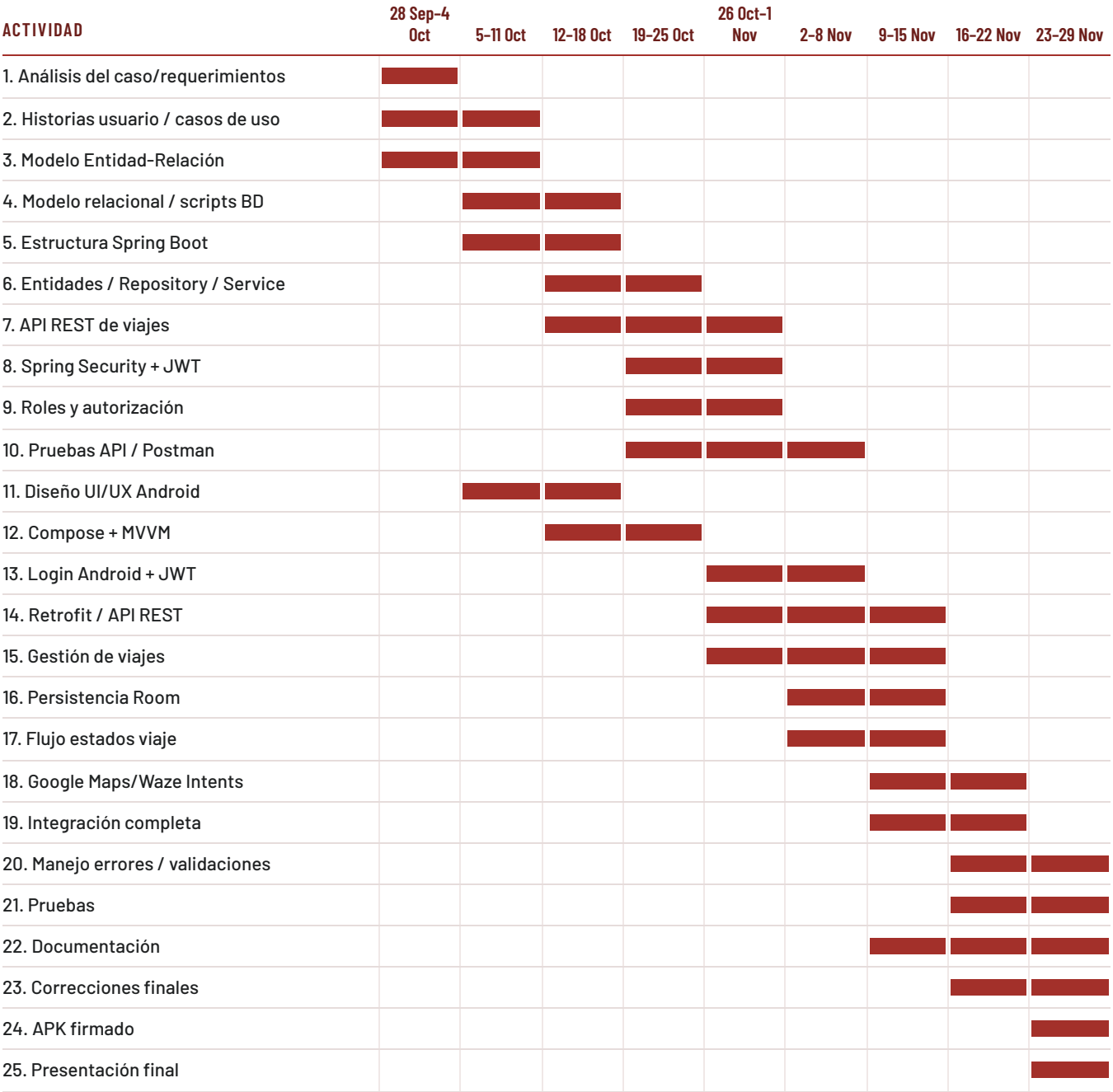
Geolocalización

El caso exige direcciones **georreferenciadas mediante coordenadas de prueba**, no necesariamente capturar permanentemente la ubicación GPS real del conductor.

RESTRICCIONES IMPORTANTES

Como lo menciona el documento, la aplicación debe funcionar en todo momento con datos ficticios, datos que cada equipo tendrá que crear. El documento menciona que tanto los datos como el backend podrían ser entregados. **Eso no es así, cada equipo debe crear absolutamente todo.**

CRONOGRAMA



ALCANCE FUNCIONAL MÍNIMO

Base de datos	MER + modelo relacional + implementación.
Backend	Spring Boot + JPA + API REST + Spring Security + JWT + roles.

Android	Kotlin + Compose + Material Design 3 + MVVM + Retrofit + Room.
Funcional	autenticación, creación de solicitudes, consulta de viajes, asignación de conductor, estados Pendiente → Asignado → En Curso → Finalizado, consulta de viajes del conductor y navegación mediante Google Maps/Waze.
Ingeniería	Git/GitHub, validaciones, manejo de errores, pruebas, documentación y APK firmado.